

Linux - Part 3

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

Linux - SRE Production Telemetry & Incident Diagnostics (Part 3)

1. Topic Introduction

This section covers critical production telemetry, real-time diagnostics, kernel call tracing (e.g. using `perf` and flamegraphs), cascading congestion events, and scenario-based incident runbooks.

2. Why This Topic is Critical

At scale, issues manifest as complex, multi-variable incidents. Senior engineers must trace microsecond call stacks, interpret lock contentions, and configure precise thresholds.

3. High-Telemetry Profiling & Call Tracing

- **`perf` Tracing**: Captures kernel call stacks and traces CPU cycles.
- **Flamegraphs**: Visually maps slow code blocks, showing hot-spots in memory or VFS operations.

4. Hardened Monitoring Alerts

Below is a production Prometheus Alerting Rule to identify read-only filesystem errors:

```
groups:
- name: HostStorageAlerts
  rules:
  - alert: HostFilesystemReadonly
    expr: node_filesystem_readonly{fstype=~"ext4|xfs"} == 1
    for: 0m
    labels:
      severity: critical
    annotations:
      summary: "Filesystem became read-only on {{ $labels.instance }}"
      description: "Kernel detected VFS write errors, mounting drive as read-only to protect blocks."
```

Linux - Production SRE & Diagnostics (Q41 to Q50)

Q41. How do you troubleshoot high 'iowait' system bottlenecks?

Detailed Answer: High ``iowait`` indicates that CPU cores are sitting idle waiting for outstanding disk or block device I/O operations to complete. First, run ``vmstat 1`` to check the wait queue (``b`` column). Next, run ``iostat -xz 1`` to identify which disk partition has high utilization (``%util``) and wait times (``await``). Finally, run ``iotop`` to pinpoint the specific process ID causing the I/O storm.

Production Scenario: A backend database experiences write latency spikes. Running ``iostat`` shows disk utilization at 99.8% and write wait time ``w_await`` at 180ms. ``iotop`` reveals a wild analytics run writing temp tables to disk, prompting the SRE to kill the analytics query.

Q42. What causes the kernel to mount a filesystem as Read-Only (RO) in production?

Detailed Answer: The virtual file system (VFS) mounts a filesystem as read-only when it detects underlying hardware errors, SCSI command timeouts, or journaling block corruption. This safety mechanism protects data from further corruption.

Production Scenario: A cloud VM loses contact with its SAN storage volume. The Linux kernel logs "SCSI error - dead device" and instantly remounts the root disk as read-only. Web processes fail on log writes. SREs recover by inspecting ``dmesg``, mounting recovery disks, or replacing the storage link.

Q43. How do you profile kernel hot-spots under high load using Flamegraphs?

Detailed Answer:

1. Record CPU call stacks: ``sudo perf record -F 99 -a -g -- sleep 30``.
2. Script the profile: ``sudo perf script > out.perf``.
3. Generate the visual map using FlameGraph scripts to find which kernel call paths occupy the widest horizontal space.

Production Scenario: An API gateway saturates CPU. Generating a Flamegraph reveals that 45% of CPU time is spent on netfilter packet routing evaluations, leading SREs to optimize iptables rules.

Q44. What is context switching overhead, and how do you diagnose it?

Detailed Answer: Context switching occurs when the kernel scheduler suspends one thread on a CPU core to load another. This requires copying CPU registers and invalidating L1 cache tables. High switching rates slow down processing. It is diagnosed using ``vmstat 1`` (look at ``cs`` column).

Production Scenario: An application spawns 1000 active execution threads on a 2-core VM. ``vmstat`` shows context switches (``cs``) spiking to 250,000/sec. CPU ``sy`` space spikes, starving useful code. SREs resolve by modifying thread configurations to pool and limit processes.

Q45. Explain how socket buffer depletion drops packets before reaching the TCP stack.

Detailed Answer: When high network packet spikes hit a network card, they are placed in kernel socket queues (SND/RCV buffers). If processes cannot read from the socket queue fast enough, the buffers fill up and incoming packets are dropped.

Production Scenario: A high-concurrency microservice drops connection requests during peaks. Running ``ss -s`` or ``netstat -s`` reveals "packet receive errors" increasing. SREs resolve by tuning ``net.core.rmem_max`` and adjusting TCP buffer margins.

Q46. How do you debug memory fragmentation issues inside JVM host nodes?

Detailed Answer: Memory fragmentation prevents the kernel from allocating contiguous blocks. Trace allocations in ``/proc/buddyinfo`` and use ``sysctl -w vm.compact_memory=1`` to force contiguous block compactions.

Production Scenario: A JVM host node dumps page allocation failures in syslogs despite showing 40% total free RAM, causing memory compaction runs that block GC threads and trigger microservice latency spikes.

Q47. Explain how lock contention ('spinlocks') saturates CPUs under heavy network forwarding.

Detailed Answer: Spinlocks cause CPU cores to continuously poll a locked memory address in a tight loop instead of sleeping. When multiple cores compete for a single queue lock, they saturate CPU in System (``sy``) space.

Production Scenario: A packet router handling millions of packets/sec shows CPU core usage at 100% in system mode. Profiling with ``perf`` shows ``_raw_spin_lock`` occupying 60% of CPU time, indicating interface queues must be split.

Q48. What is the impact of ARP cache overflows on cluster network communications?

Detailed Answer: The ARP table stores IP-to-MAC maps. When cluster hosts exceed the ARP threshold (``net.ipv4.neigh.default.gc_thresh3``), new entries are blocked, dropping outgoing network packets.

Production Scenario: A newly scaled Kubernetes cluster of 2,000 nodes loses connection to standard resources. ``dmesg`` logs "neighbor table overflow". Resolving requires raising ``gc_thresh1/2/3`` limits to handle larger host counts.

Q49. Formulate a Prometheus alerting rule to detect dirty memory page writeback saturation.

Detailed Answer: Monitor the rate of dirty page backlogs relative to disk write limits by tracking disk queue saturation and dirty page trends:

```
- alert: HighKernelDirtyPages
  expr: (node_memory_Dirty_bytes / node_memory_MemTotal_bytes) * 100 > 8
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High dirty page backlog on {{ $labels.instance }}"
    description: "Dirty pages exceed 8% of total RAM, threatening filesystem sync blocks."
```

Production Scenario: Alerting on this allows SREs to proactively detect disk storage slow-downs before dirty flushes throttle database writes.

Q50. How do you diagnose a system with a load average of 150 but CPU utilization at only 5%?

Detailed Answer: Under Linux, the load average counts processes in two states: Runnable (R) and Uninterruptible Sleep (D). Low CPU utilization with elevated load averages indicates that almost all 150 processes are blocked in the 'D' state waiting for storage, disk queuing, or NFS mounts.

Production Scenario: An SRE logs in during an outage. CPU is at 2%, but load average is at 180. Running ``vmstat 1`` reveals the ``b`` (blocked) column shows 150 processes. Checking ``dmesg`` confirms storage connection failures, identifying disk block outages.